3 ways of selecting

SELECT * All columns on the table

SELECT DISTINCT column
FROM table | different tables

SELECT column1
, column2
, column3
FROM table_name;

Data Filtering ~~way to~~

Is where filter data if we want to get specific information from our data.
Is ~~SELECT~~ excluding other things eg excluding other years (2024/2025) only focus on 2026.

Is a way to exclude rows of data that do not meet a specific criteria.

You want specific information

WHERE statement

Syntax: SELECT column1,
column2,
column3,
FROM table_name
WHERE column condition;

students results continue

SELECT *
FROM ~~re~~ students results
WHERE marks 760;

student_ID	name	marks
101	Rohit	99

- reduced table
- Filtered

Databricks

SQL Operators

1. Arithmetic Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division

2. Comparison Operators

Comparison Operators	Description
=	equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
<>	Not equal to

You want specific information

WHERE Statement

Syntax: SELECT column1,
column2,
column3,
FROM table_name
WHERE Column condition;

Students results continue

SELECT *
FROM ~~re~~ students_results
WHERE marks 760;

student_ID	name	marks
101	Roshi	99

- reduced table
- Filtered

Databricks

3. Logical Operators

Operator	Description
AND OR	Both conditions must be met or one condition must be met.
BETWEEN	must be within the range
IN	must be contained in the list
NOT	It is a opposite result of what is specified.
LIKE	Looks for a pattern.

Operators → multipurpose tool

Data filtering → WHERE Statement
→ filtering statement used to filter our data based on specific conditions

We can use SQL operators to build the conditions for filtering our data

Syntax (Traditions) — SELECT Column1
Column2
Column3

FROM table_name
WHERE Condition;

Users table

column name

User_Id	User_name	Country/City
1	Nomvula	JHB
2	David	Cape Town
3	Anele	Durban
4	Kabelo	Pretoria
5	Erato	P.E

write SQL query to give nam of users in Durban value

SELECT *
FROM Users ~~table~~
WHERE city = 'Durban'

condition - we used the operators to built the condition

User-Id	User-name	City
~~4~~ 3	~~Kabelo~~ Anele	Durban

membership_Plan

plan_id	plan_name	monthly_price
101	Basic	79
102	Standard	129
103	Premium	199
104	Family Plan	249
105	mobile	~~249~~ 150

monthly ~~plan~~ ~~that~~ cost more than 150
write an SQL

```
SELECT   plan_name
         monthly_price
FROM     membership_plan
WHERE    monthly_price > 150
```

Plan_name	Monthly_price
Premium	199
Family	249

151, 152, 153 etc
150 excluded

Monthly_price ≥ 150
(150, 151, 152, 153) 150 included

BETWEEN - An inclusive statement

```
SELECT *
FROM table_name
WHERE column_name (BETWEEN
    (lower AND upper
BETWEEN 5 AND 7
(5,6,7)
```

Databricks

SQL editor - New query
Run Selected

SQL Logical Operators

AND Statement

- Allows to combine multiple conditions.
- All the conditions must be satisfied
- We commonly use the AND statement when building logical conditions for filtering data

```
SELECT column1
                || 2
E.g.,           ||
FROM  table_name
WHERE  Condition 1 AND condition 2
  AND Condition 3 .. ...
```

Students

Student_Id	name	course	Age	City
1.	Thabo	Data Analytics	22	JHB
2	Sarah	Data Science	19	Pretoria
3	John	Analytics Data	25	Durban
4	Lerato	Data Analytics	20	JHB
5	Musa	Data Science	23	JHB

Write an SQL query to give the names of students living in JHB that are doing data Analytics

```
SELECT *

FROM students

WHERE city = 'JHB' AND course = 'Data Analytics';
```

condition 1 — condition 2

outcome

Student_id	name	course	age	city
1	Inabo	Data Analytics	22	JHB
2	Lerato	Data Analytics	20	JHB

OR Statement
- combines multiple conditions
- One of the condition must be met

Eg (use previous page table)
Write an SQL to give:
Student names living in JHB OR doing data analytics

```
SELECT *
FROM Student
WHERE city='JHB' OR course
'Data analytics';
```

Outcome
Thabo
John
Lerato
Musa

IN Statement
- It is a way of filtering data by specifying the values of interest from a specific column.

Syntax: SELECT *
FROM students
WHERE column IN (value1, value2, (value3, ---- etc)
- No limit in the number of values you can specify.

Use same table

write an SQL requery to give people living in Durban & Pretoria

```
SELECT name, city
FROM students
WHERE city IN('Durban', 'Pretoria');
```

academic

Name	City
Sarah	Pretoria
John	Durban

NOT statement

```
SELECT name,
       City
FROM Students
WHERE city NOT IN ('Durban', 'Pretoria');
```

Exclusion statement

opposite results of what you ask for

JHB

Name	City
Tivabo	JHB
Lerato	JHB
Musa	JHB

BETWEEN Statement

- Allows to filter data based on specific range of values.
- It is an inclusive statement. This means that the lower and upper bounds of the Between statement are inclusive.

```
SELECT column1 (name)
       || 2
       || 3
          (Students)
FROM table_name
          (AGE) BE (20 AND 22)
WHERE : Column . BETWEEN lower bound

    AND . Upper bound
```

Use same table; write an SQL query to give name & age of students btwn ages of 20 to 23

SELECT name
Age
FROM Students

WHERE Age BETWEEN 20
AND 23;

name ~~table~~	Age
Musa	23
Lerato	20
Thabo	22

SORTING Statement 16/06/26
ORDER BY Statement
Up In SQL this a sorting
statement

Two way of sorting data.
Up we can sort our data either
in Ascending Order (ASC) or
Descending Order (DESC)

Sorting does not remove or add anything new into our data - it just re-arrange the rows in our data.

Ascending - Going UP (A-Z) - From lowest to highest

Descending - Going down
- 9-1
- Z-A
- From biggest to Smallest
- highest - lowest

Syntax:

```
SELECT *
FROM table_name
WHERE condition
ORDER BY column_name ASC;
```

for Ascending - ASC; OR DESC; for Descending

Students table

Ascending:

```
SELECT name,
       Age
name
  FROM students
ORDER BY Age ASC;
```

Sarah	19
Lerato	20
Thabo	22
Musa	23
John	25

Age

Desserding

```
SELECT name,
       Age
FROM Studens
ORDER BY Age DESC;
```

name	Age
John	25
Thabo	23
Musa	22
Thabo	20
Lerato	19
Sarah	

Aggregate Functions

Summarize data # Consilidate data

- Is used to perform a calculation on rows of a specific column and returns a single value/ a single value per group

Function	Description
① Count ( )	counts the number of rows identifies e.g. IDs (unique)
② Sum ( )	Calculates the total of a numeric column e.g. revenue
③ AVG ( ) e.g. revenue	Calculate the average of a numeric colums
④ MAX ( )	Find the maximum value in a column
⑤ MIN ( )	Finds the minimum value in a column

Employees

Emp_id	Name	Salary
101	John	R20
102	mary	R40
103	David	R35

SELECT MIN(Salary) AS min_salary

FROM employees;

min_salary
R20

SELECT MAX(salary) AS max_salary

FROM employees

max_salary
40

AS (statement)

AS Function

↳ A way to rename a column name after applying a Function

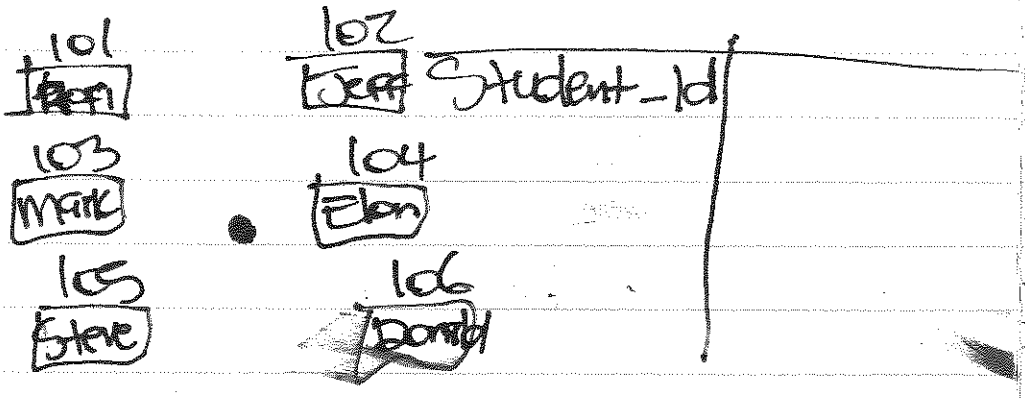
Nickname / stage name

AS = Aliases.

count

Grade 10 th,

Student



Student

Student_Id	Name	Subject
101	Rofi	Science
102	Jeff	Commerce
103	Mark	Commerce
104	Elon	Science
105	Steven	History
106	Donald	Commerce

SELECT COUNT(DISTINCT student_id)
AS number_of_students

FROM students;

number of students
6

Counting the number of student in the class table

GROUP BY — when categorical column that is not aggregated in your SELECT statement

SELECT categorical column

WHERE vs HAVING — FILTERS

Filter rows before grouping

groups after grouping

Databricks class 18/06/26

RUN ALL - CTRL + ENTER

Use Kaggle - to code practice

Limit Statement — does not filter anything, just limit number of rows.

- It is a way of restricting the number of rows you want in your final output.

eg | No. employees table

Id	Name
101	Rathi.
102	Mballi
103	Zuma

```
SELECT *
FROM employees
LIMIT 2;
```

Id	Name
101	Rathi.
102	Mballi

Aggregate functions 18/06/26

#Summarize data
#Perform calculations on multiple rows of a specific column and returns a single value/ single value per group

#Consolidate data

1) Count() - count unique identification (ID)
2) Sum() - Sum numeric data (revenue)
3) AVG() - average numeric data (revenue)
4) MIN() - Looks for min value in a column
5) MAX - Looks for max value in a column

GROUP BY Statement

(we group by a column that is not aggregated

SELECT categorical column - not numeric
- gender
- race

Aggregate - function column
- numerical column

HAVING Statement

↳ It is used to filter data that is aggregated.

↳ We apply the HAVING to data that is already aggregated

It is used to filter grouped results after using a GROUP BY

WHERE filters rows before group (filter ~~before~~ un aggregated) data

HAVING filters groups after groupings (aggregated)

Sales

Cust-Id	Cust_name	Product	Amount
101	Rafhi	Laptop	5000
101	Rafhi	Mouse	3000
102	Abel	Keyboad	700
102	Abel	Monitor	2500
103	Ciara	mouse	300

write an SQL query to find customers who spent more than R3000

```
SELECT customer_name,
       SUM(amount) AS total_spent
FROM sales
GROUP BY cust_name,
HAVING SUM(amount) > 3000
```

Cust_name	Total_spent
Rathi	5300
Abel	3200
Clara	300

SUM(Adding different values in the same groups

SELECT Department IT

Sum(salary) AS total salary

FROM employees

-- SELECT Sum(salary) AS
total_salary_paid

FROM employees

WHERE department = 'IT'

SELECT Department
FROM employees
GROUP BY Salary

SELECT AVG(salary)
department
FROM employees AVG Salary
GROUP BY department ORDER BY 1
DESC